Lab 1

Installation of Virtual Box and Ubuntu

In order to practice the C code in Linux environment, you are required to install Virtual Box and Ubuntu operating system on your systems using the guidelines given below.

Solution:

Following are the guidelines and helping material including required software and their installations:

Step No.1
To install the Linux first you have to install Virtual Box visualization software in your Computer system.

Follow the tutorial to download Virtual Box form the following link:
https://vulms.vu.edu.pk/Courses/CS604/Downloads/CS604-VirtualBox.mp4
(You can use VMware as well.)

Step No.2
Download Ubuntu Linux from the following link:
http://de.releases.ubuntu.com/16.10/ubuntu-16.10-desktop-i386.iso

Follow these tutorials to install Ubuntu Linux in Virtual Box:
1 - https://vulms.vu.edu.pk/Courses/CS604/Downloads/CS604-Virtual%20Machine.mp4
2 - https://www.lifewire.com/run-ubuntu-within-windows-virtualbox-2202098

Lab 2

2.1 Linux commands for managing directories and files

Practice the following Linux commands in Shell:

1. Write a command to show your current working directory.
2. Write a command to change your current directory to Desktop.
3. Write a command to create a directory structure as: CS604/Files/CFiles
4. Write a command to display the contents of Desktop.
5. Write a command to remove a directory named CS604.

Solution:

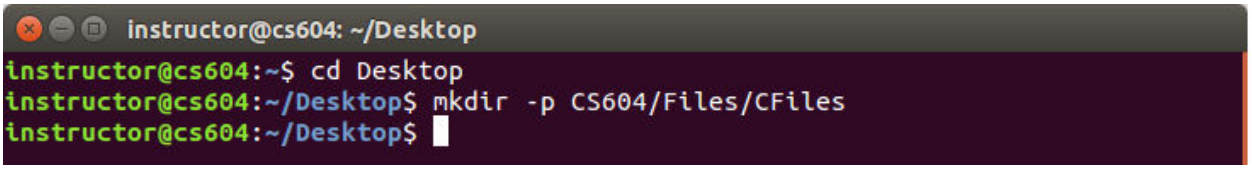
1. Write a command to show your current working directory.



2. Write a command to change your current directory to Desktop.



3.Write a command to create a directory structure as: CS604/Files/CFiles



4. Write a command to display the contents of new created directories.

```
instructor@cs604: ~/Desktop
instructor@cs604:~$ cd Desktop
instructor@cs604:~/Desktop$ ls
A                exercise.c     myInfo.c
a1               FCFS           myInfo.c~
a1.c             FCFS.c         myprogram.c
a1.c~            FCFS.c~        OS
A.c              lab2           Q1
A.c~             lab2.c         Q1.c
a.out            lab2.c~        Q1.c~
Ass1             my             SPRING 2018_CS304_1_BC170405960.docx
Ass1.c~          myexercise.c   ThreadFile
BC123456789      myFile         ThreadFile.c
BC123456789.c    myfile1.c      ThreadFile.c~
BC123456789.c~   myFile.c       Untitled Document
bc170400674      myFile.c~      Untitled Document~
CS604            myInfo
instructor@cs604:~/Desktop$
```

5. Write a command to remove a directory named CS604.

```
instructor@cs604: ~/Desktop
instructor@cs604:~$ cd Desktop
instructor@cs604:~/Desktop$ rmdir CS604
instructor@cs604:~/Desktop$
```

2.2 Compiling and running C code in Linux

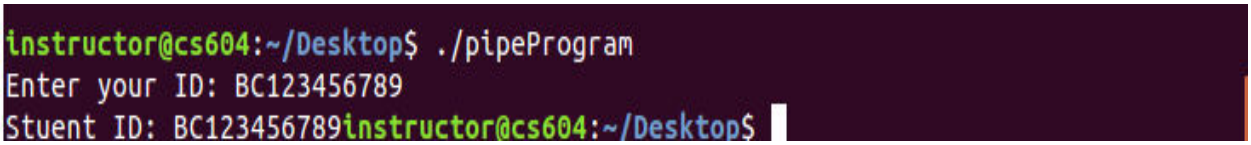
Write a program in C which will display your student id and name as output. Compile and execute your program in Command prompt to display the output as shown below.

```
instructor@cs604: ~/Desktop
instructor@cs604:~$ cd Desktop
instructor@cs604:~/Desktop$ gcc myInfo.c -o myInfo
instructor@cs604:~/Desktop$ ./myInfo
Student ID: BC123456789
Student Name: ABC
instructor@cs604:~/Desktop$
```

Lab 3

Inter-process communication between processes using PIPES

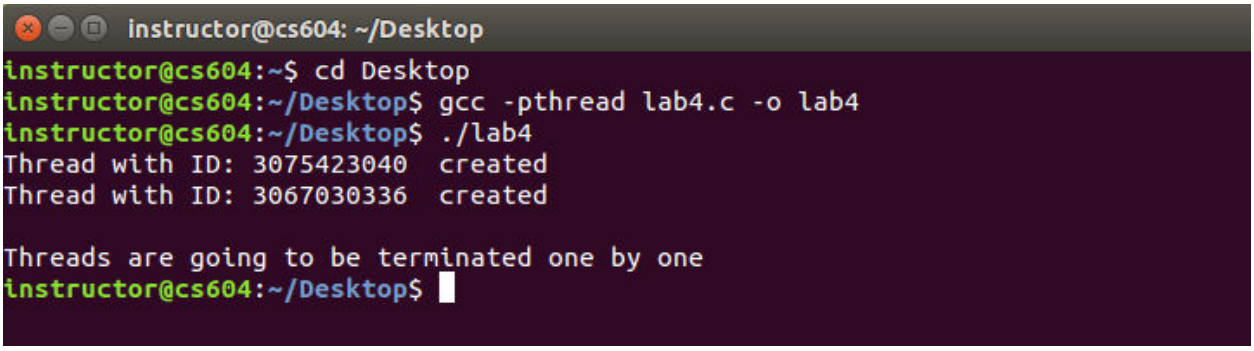
Write a program in C for inter-process communication between child and parent processes. You have to use pipe() system calls for creating a pipe between parent and child processes. The child will write the student id trough PIPE and parent will read the student id from pipe and display it on screen. Parent will wait for child process until child write data to pipe. Compile and execute your program in Command prompt to display the output as shown below.



Lab 4

Threads creation and termination

Write a program in C that will create two threads named as Thread 1 and Thread 2. You are required to run these threads in parallel fashion and display the Thread IDs according to the output as shown below. At the end, all threads will be terminated. Compile & run C program on Linux Operating system.



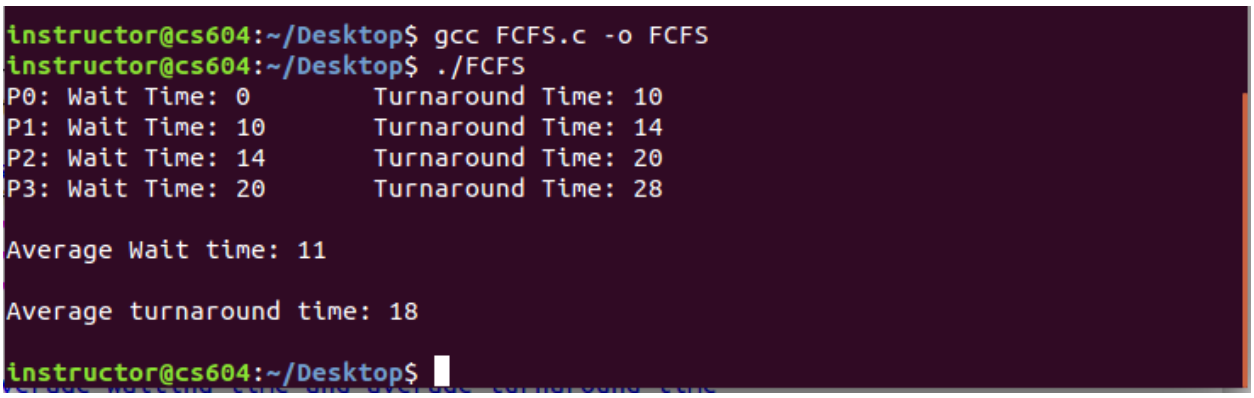
Lab 5

Implementing FCFS scheduling algorithm in C

Write a program in C to implement FSFS scheduling algorithm. Given the list of Processes and their burst time, calculate waiting time and turnaround time for each process. Also calculate average waiting time and average turnaround time. For example, the list of processes and their CPU burst time are as follows:

Processes	Burst Time
P0	10
P1	4
P2	6
P3	8

Assume that all the processes arrive at time 0 in sequence P0, P1, P2 P3



Solution:

```
#include <stdio.h>

int main(){

int n = 4 ;          // number of processes
int bTime[4] ;    // array for storing burst time for each process
int wTime[4] ;  // array for storing wait time for each process
int tATime[4] ; // array for storing turnaround time for each process
bTime[0] = 10 ;
bTime[1] = 4 ;
bTime[2] = 6 ;
```

```
bTime[3] = 8 ;

int avgWTime = 0 ;
int avgtATime = 0 ;
wTime[0] = 0 ;

// loop for calculating waiting time for each process
for(int i = 1; i<n; i++){
        wTime[i] = wTime[i-1]+bTime[i-1] ;      // calculating wait time for each process
        avgWTime = avgWTime + wTime[i] ;    }

// loop for calculating turnaround time for each process
for(int i = 0; i<n; i++){
        tATime[i] = wTime[i]+bTime[i] ;          // calculating turnaround time for each process
        avgtATime = avgtATime + tATime[i] ;
}

avgWTime = avgWTime / n;    // calculating average waiting time
avgtATime = avgtATime / n ;  // calculating average turnaround time

// loop for displaying waiting time and turnaround times
for (int i =0; i<n; i++){
        printf("P%i: " , i) ;
        printf("Wait Time: %i\t", wTime[i]) ;              // printing wait time for each process
        printf("Turnaroung Time: %i\n", tATime[i]) ; // printing turnaround time for each
process
}

// displaying average waiting time and average turnaround time
printf("\nAverage Wait time: %i\n", avgWTime) ;
printf("\nAverage Wait time: %i\n\n", avgtATime) ;

} // end of main()
```

Lab 6

Implementing Round Robin scheduling algorithm in C

Write a program in C to implement Round Robin scheduling algorithm. The time slice for each process is 5 units. Given the list of Processes and their CPU burst time, calculate waiting time and turnaround time for each process. You also have to calculate average waiting and average turnaround time and display their values on screen.

For example, the list of processes and their CPU burst time are as follows:

Processes	Burst Time
P0	12
P1	9
P2	4
P3	6

Time slice = 5 units

Assume that all the processes arrive at time 0 in sequence P0, P1, P2 P3

Output of program:

```
instructor@cs604:~/Desktop$ gcc RR.c -o RR
instructor@cs604:~/Desktop$ ./RR

Turnaround Time: 31
Turnaround Time: 28
Turnaround Time: 14
Turnaround Time: 29
Waiting Time: 19
Waiting Time: 19
Waiting Time: 10
Waiting Time: 23
Average Turnaround Time: 25
Average Waiting Time: 17
instructor@cs604:~/Desktop$
```

Solution:

```
#include <stdio.h>

int main(){

int n = 4 ;
int avgWTime = 0, avgTTime = 0 ;
int timeSlice= 5  ;
int tSlice[4] ;
int bTime[4] ;
int tATime[4];
int wTime[4];
int count[4] ;

bTime[0] = 12 ;
bTime[1] =9 ;
bTime[2] = 4 ;
bTime[3] = 6 ;
int value[4] ;

for (int i = 0 ; i<n ; i++ ){
          count[i] = 0 ;
          value[i] = 0 ;
}

int flag ;
int counter = 0;

do{

        flag = 0 ;
        for ( int i = 0 ; i<n; i++ ){
            if (bTime[i] < timeSlice || bTime[i] == 0)
                    tSlice[i] = bTime[i];
            else
                    tSlice[i] = timeSlice ;
            for (int j = 0 ; j<tSlice[i]; j++ ){
```

```
                bTime[i]-- ;
        counter++ ;
          }

          tSlice[i] = bTime[i];
          if(value[i] != 1)
          {
                count[i] = counter ;
          }
      }

     for ( int i = 0 ; i<n; i++ ){
        if (bTime[i] != 0){
                flag = 1 ;
       }
        else
                value[i] = 1;
   }

}while (flag != 0);

bTime[0] = 12 ;
bTime[1] = 9 ;
bTime[2] = 4 ;
bTime[3] = 6 ;

for (int i = 0 ; i<n ; i++ ){
        wTime[i] = count[i] - bTime[i] ;
}

printf("\n");

// for loop to display turnaround time for each process
for (int i = 0 ; i<n ; i++ ){
        printf("Turnaround Time: %i\n", count[i]) ;
}

// for loop to display wait time for each process
```

```
for (int i = 0 ; i<n ; i++ ){
        printf("Waiting Time: %i\n", wTime[i]) ;
}

// for loop to calculate average wait time
for (int i = 0 ; i<n ; i++ ){
    avgWTime = avgWTime+wTime[i] ;

}

// for loop to calculate average turnaround time
for (int i = 0 ; i<n ; i++ ){
    avgTTime = avgTTime+count[i] ;
}

printf("Average Turnaround Time: %i\n", avgTTime/n) ;
printf("Average Waiting Time: %i\n", avgWTime/n) ;

} // end of main()
```

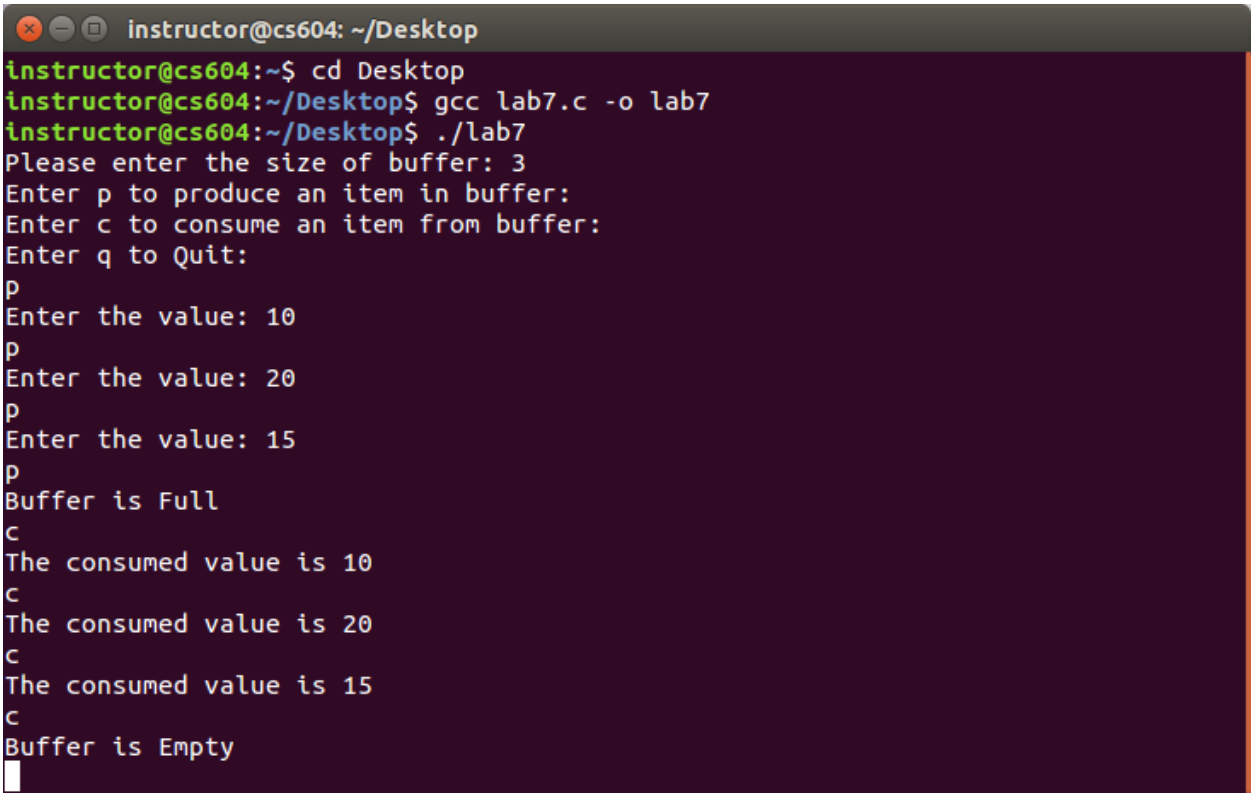
Lab 7

Producer consumer problem for Process synchronization

Write a C program to implement Producer consumer problem for process synchronization. A producer process produces information that is consumed by a consumer process. You have to implement the producer-consumer problem by using shared memory like buffer that is shared by the producer and consumer processes. A producer can produce one item while the consumer is consuming another item. The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced.

In a program, the user will take size of buffer from user and then producer process will produce items in buffer and consumer process will consume items from buffer and display its values on screen.

The output of program is as shown below:



Solution:

```
#include<stdio.h>

int main()
{
int bufsize, in, out, produce = 0, consume = 0 , counter = 0;
in = 0; out = 0;

printf("Please enter the size of buffer: ");
scanf("%d", &bufsize);

int buffer[bufsize] ;

char choice ;

printf ("Enter p to produce an item in buffer:\n") ;
printf ("Enter c to consume an item from buffer:\n") ;
printf ("Enter q to Quit:\n") ;

do{

scanf("%c", &choice);

switch(choice){
case 'p':

// code for producer

if(counter == bufsize)
printf("Buffer is Full\n");
else { printf("Enter the value: ");
scanf("%d", &produce);
buffer[in] = produce;
 in = (in+1)%bufsize;
counter++ ;
}
break ;

// code for consumer
case 'c':

if(counter == 0)
printf("Buffer is Empty\n");
```

```
 else {
 consume = buffer[out];
printf("The consumed value is %d\n", consume);
 out = (out+1)%bufsize;
counter-- ;
}
break ;

} // end of switch
}while( choice != 'q') ;

}
```

Mechanism to Conduct Lab:

Students and teacher will communicate through google meet. Student will edit, compile and run the C code for a given problem in Linux environment and share the screen with teacher.

Lab 8: Implementing Dining Philosophers Problem in C

Write a C program that solves Dining philosopher's problem. Suppose there are five Philosophers whose work is just thinking and eating. They sat at a round table for dinner. To complete dinner each must need two chopsticks (or forks). But there are only five chopsticks available (chopsticks always equal to number of philosophers) on table. They take in such a manner that, first take left fork and next right fork. But problem is they try to take at same time. Since they are trying at same time, Fork 1, 2, 3, 4, 5 taken by Philosopher 1, 2, 3, 4, 5 respectively (since they are left side of each). And each one tries to take right side Fork. But no one found available Fork. Moreover, each one thinks that someone will release the Fork and then I can eat. This continuous waiting leads to Deadlock situation.

To solve this deadlock situation, Last philosopher (any one can do this) first try to take right side fork and then left side fork. i.e in our example 5th person tries to take 4th Fork instead of 5th one. Since 4th Fork already taken by 4th the person, he gets nothing. But he left 5th Fork. Now the first person will take this 5th Fork and complete dinner and make 1st and 5th available for remaining people. Next 2nd person takes 1st fork and completes and releases 1st and 2nd. This continues until all finishes dinner. In Operating System, this concept is used in process synchronization. Same problem but instead of Philosophers processes are there and instead of Forks, Resources are there. We follow above solution to avoid dead lock condition.

The output of program is as shown below:

```
haseebakmal@ubuntu: ~/Desktop
haseebakmal@ubuntu:~/Desktop$ gcc Lab8.c -o Lab8
haseebakmal@ubuntu:~/Desktop$ ./Lab8
Fork 1 taken by Philosopher 1
Fork 2 taken by Philosopher 2
Fork 3 taken by Philosopher 3
Philosopher 4 is waiting for fork 3

Till now num of philosophers completed dinner are 0

Fork 4 taken by Philosopher 1
Philosopher 2 is waiting for Fork 1
Philosopher 3 is waiting for Fork 2
Philosopher 4 is waiting for fork 3

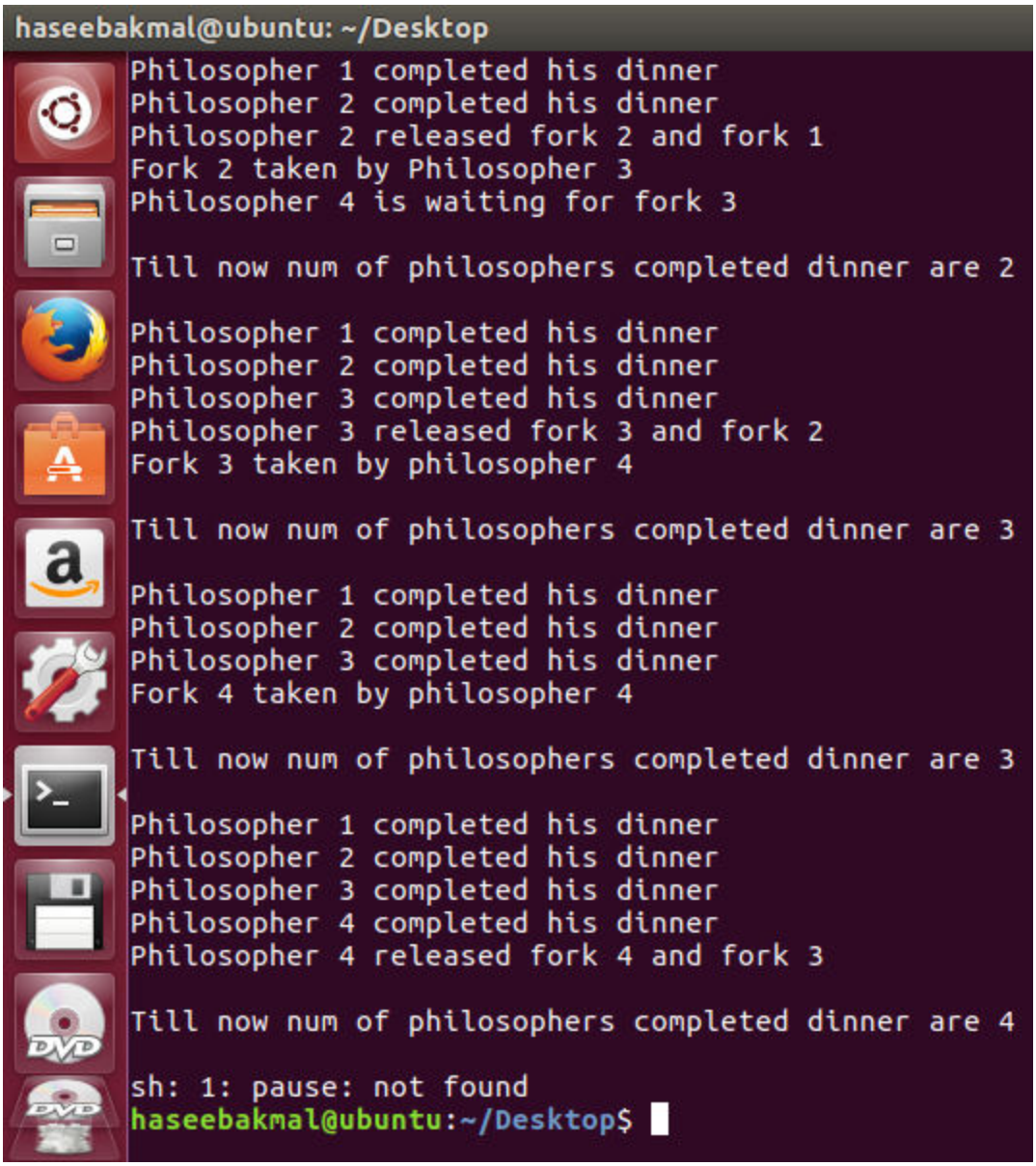
Till now num of philosophers completed dinner are 0

Philosopher 1 completed his dinner
Philosopher 1 released fork 1 and fork 4
Fork 1 taken by Philosopher 2
Philosopher 3 is waiting for Fork 2
Philosopher 4 is waiting for fork 3

Till now num of philosophers completed dinner are 1

Philosopher 1 completed his dinner
Philosopher 2 completed his dinner
Philosopher 2 released fork 2 and fork 1
Fork 2 taken by Philosopher 3
Philosopher 4 is waiting for fork 3

Till now num of philosophers completed dinner are 2
```




```
#include<stdlib.h>
#include<stdio.h>
#define n 4

int compltedPhilo = 0,i;
struct fork{
   int taken;
} ForkAvil[n];

struct philosp{
   int left;
```

```
   int right;
}Philostatus[n];

void goForDinner(int philID){
   if(Philostatus[philID].left==10 && Philostatus[philID].right==10)
      printf("Philosopher %d completed his dinner\n",philID+1);
   //if already completed dinner
   else if(Philostatus[philID].left==1 && Philostatus[philID].right==1){
         //if just taken two forks
         printf("Philosopher %d completed his dinner\n",philID+1);

         Philostatus[philID].left = Philostatus[philID].right = 10;
         int otherFork = philID-1;

         if(otherFork== -1)
            otherFork=(n-1);

         ForkAvil[philID].taken = ForkAvil[otherFork].taken = 0;
         printf("Philosopher %d released fork %d and fork %d\n",philID+1,philID+1,otherFork+1);
         compltedPhilo++;
      }
      else if(Philostatus[philID].left==1 && Philostatus[philID].right==0){

            if(philID==(n-1)){
               if(ForkAvil[philID].taken==0){
                  ForkAvil[philID].taken = Philostatus[philID].right = 1;
                  printf("Fork %d taken by philosopher %d\n",philID+1,philID+1);
               }else{
                  printf("Philosopher %d is waiting for fork %d\n",philID+1,philID+1);
               }
            }else{
               int dupphilID = philID;
               philID-=1;

               if(philID== -1)
                  philID=(n-1);

               if(ForkAvil[philID].taken == 0){
                  ForkAvil[philID].taken = Philostatus[dupphilID].right = 1;
                  printf("Fork %d taken by Philosopher %d\n",philID+1,dupphilID+1);
               }else{
                  printf("Philosopher %d is waiting for Fork %d\n",dupphilID+1,philID+1);
               }
            }
```

```
        }
        else if(Philostatus[philID].left==0){ //nothing taken yet
              if(philID==(n-1)){
                 if(ForkAvil[philID-1].taken==0){
                    ForkAvil[philID-1].taken = Philostatus[philID].left = 1;
                    printf("Fork %d taken by philosopher %d\n",philID,philID+1);
                 }else{
                    printf("Philosopher %d is waiting for fork %d\n",philID+1,philID);
                 }
              }else{ //except last philosopher case
                 if(ForkAvil[philID].taken == 0){
                    ForkAvil[philID].taken = Philostatus[philID].left = 1;
                    printf("Fork %d taken by Philosopher %d\n",philID+1,philID+1);
                 }else{
                    printf("Philosopher %d is waiting for Fork %d\n",philID+1,philID+1);
                 }
              }
     }else{}
}

int main(){
   for(i=0;i<n;i++)
      ForkAvil[i].taken=Philostatus[i].left=Philostatus[i].right=0;

   while(compltedPhilo<n){
      for(i=0;i<n;i++)
         goForDinner(i);
      printf("\nTill now num of philosophers completed dinner are %d\n\n",compltedPhilo);
   }

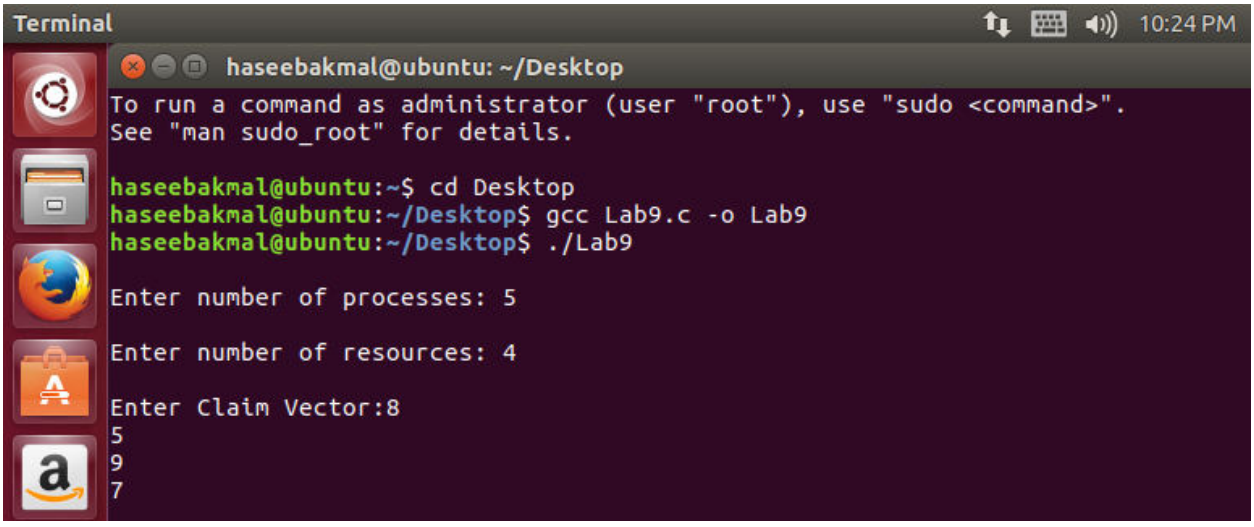
   system("pause");
}
```

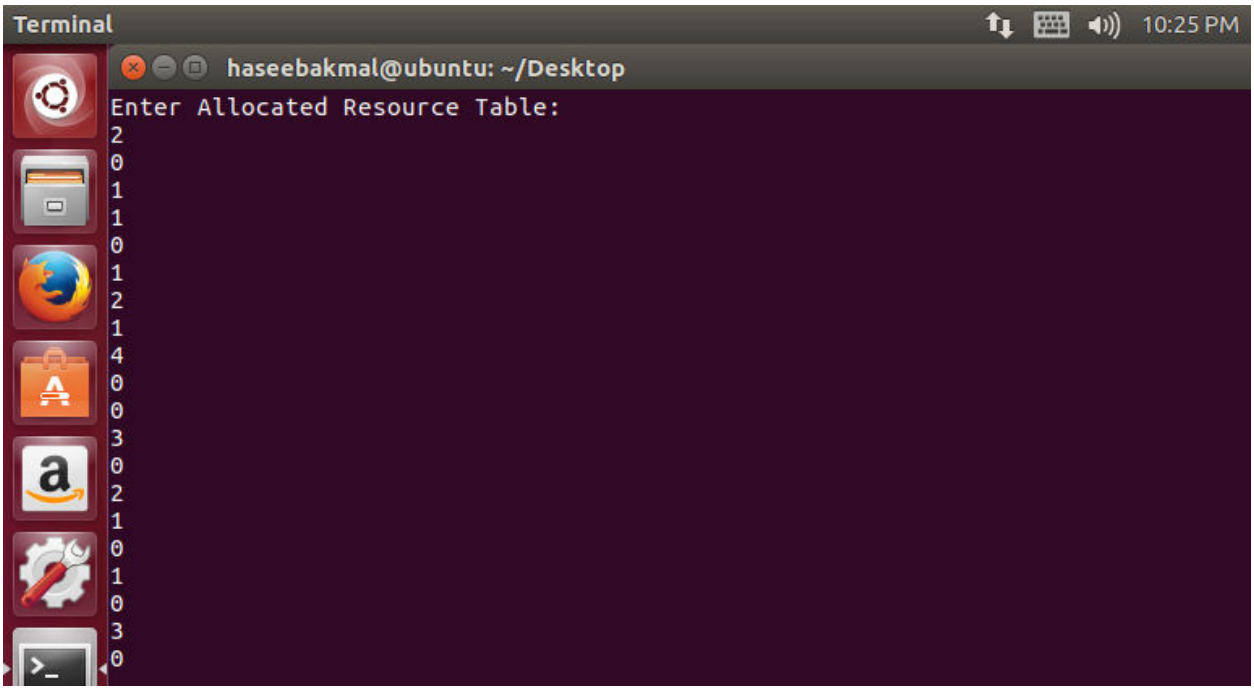
Lab 9: Implementing Banker's Algorithm for Deadlock avoidance in C

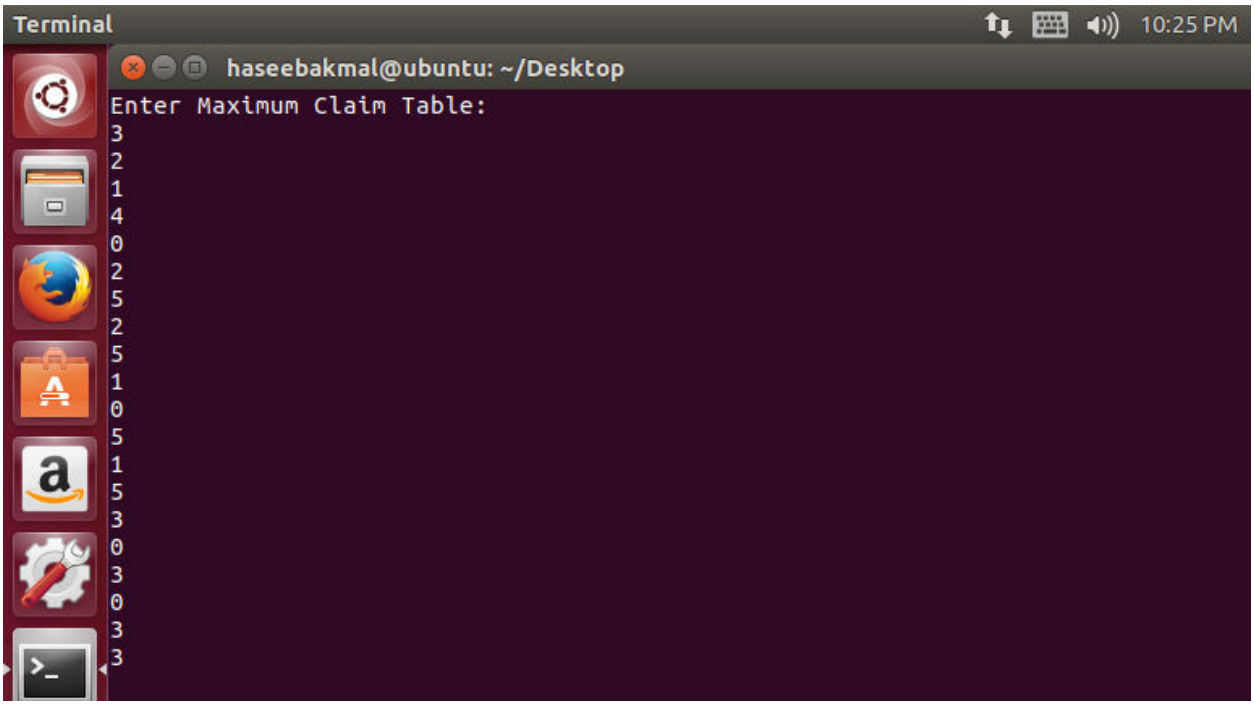
In this algorithm, when a new process enters the system, it must declare the maximum number of instances of each resource type that it may need, i.e., each process must a priori claim maximum use of various system resources. This number may not exceed the total number of instances of resources in the system, and there can be multiple instances of resources. When a process requests a set of resources, the system must determine whether the allocation of these resources will leave the system in a safe state. If it will, the resources are allocated; otherwise the process must wait until some other process releases enough resources. We say that a system is in a safe state if all of the processes in the system can be executed to termination in some order; the order of process termination is called safe sequence. When a process gets all its resources, it must use them and return them in a finite amount of time. Let n be the number of processes in the system and m be the number of resource types. We need the following data structures in the Banker's algorithm:

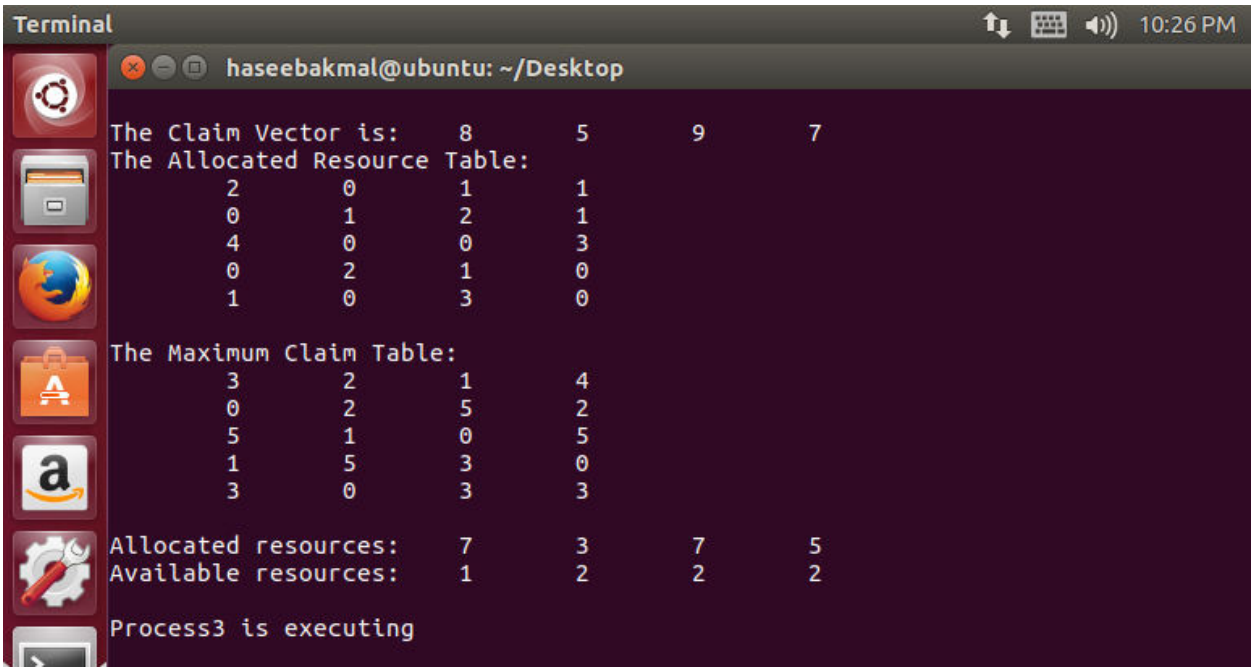
Available: A vector of length m. It shows number of available resources of each type. If Available[i] = k, then k instances of resource Ri are available.
Max: An n×m matrix that contain maximum demand of each process. If Max[i,j] = k, then process Pi can request maximum k instances of resource type Rj.
Allocation: An n×m matrix that contain number of resources of each type currently allocated to each process. If Allocation[i,j] = k, then Pi is currently allocated k instances of resource type Rj.
Need: An n×m matrix that shows the remaining resource need of each process. If Need[i,j] = k, then process Pi may need k more instances of resource type Rj to complete the task.

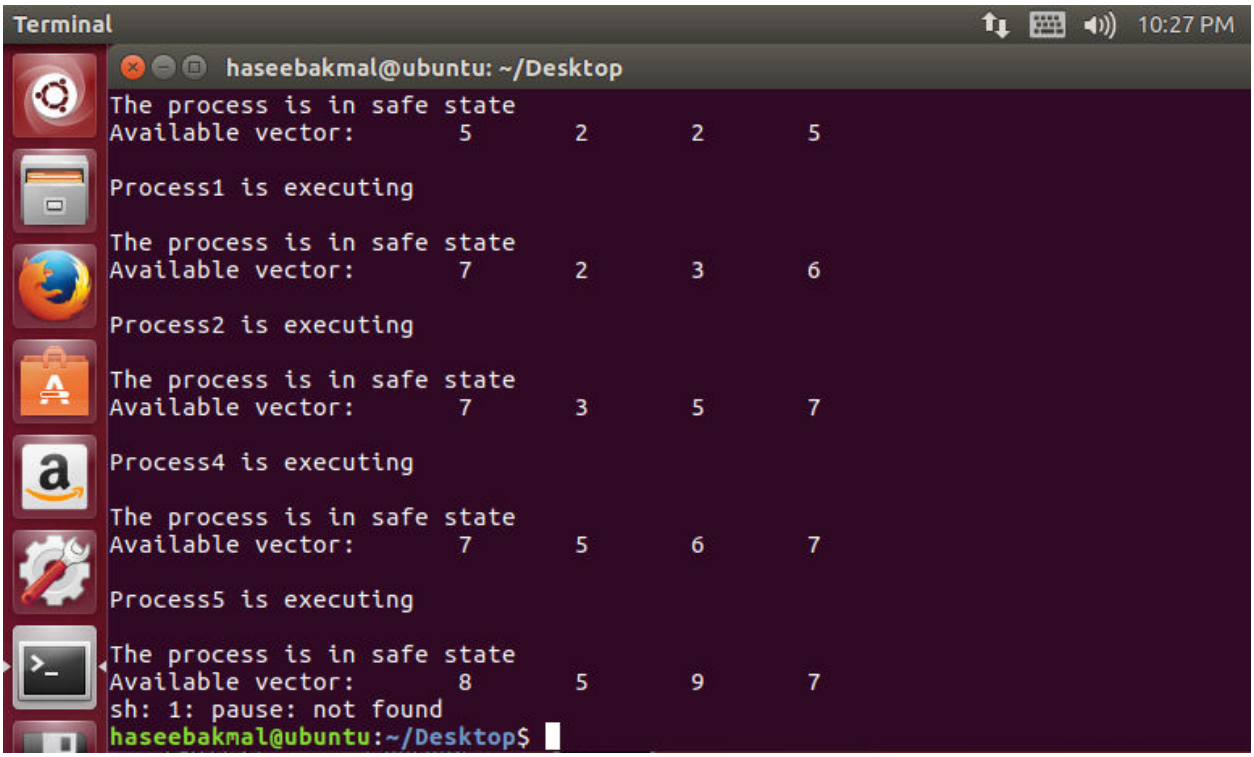
The output of program is as shown below:




Terminal
10:25 PM
haseebakmal@ubuntu: ~/Desktop
Enter Allocated Resource Table:
2
0
1
1
0
1
2
1
4
0
0
3
0
2
1
0
1
0
3
0


Terminal
10:25 PM
haseebakmal@ubuntu: ~/Desktop
Enter Maximum Claim Table:
3
2
1
4
0
2
5
2
5
1
0
5
1
5
3
0
3
0
3
3






```
#include<stdlib.h>
#include <stdio.h>
int current[5][5], maximum_claim[5][5], available[5];
int allocation[5] = {0, 0, 0, 0, 0};
int maxres[5], running[5], safe = 0;
```

```
int counter = 0, i, j, exec, resources, processes, k = 1;

int main()
{
   printf("\nEnter number of processes: ");
      scanf("%d", &processes);

      for (i = 0; i < processes; i++)
   {
         running[i] = 1;
         counter++;
      }

      printf("\nEnter number of resources: ");
      scanf("%d", &resources);

      printf("\nEnter Claim Vector:");
      for (i = 0; i < resources; i++)
   {
         scanf("%d", &maxres[i]);
      }

     printf("\nEnter Allocated Resource Table:\n");
      for (i = 0; i < processes; i++)
   {
         for(j = 0; j < resources; j++)
      {
          scanf("%d", &current[i][j]);
         }
      }

      printf("\nEnter Maximum Claim Table:\n");
      for (i = 0; i < processes; i++)
   {
         for(j = 0; j < resources; j++)
      {
               scanf("%d", &maximum_claim[i][j]);
         }
      }

   printf("\nThe Claim Vector is: ");
      for (i = 0; i < resources; i++)
   {
         printf("\t%d", maxres[i]);
```

```
}

   printf("\nThe Allocated Resource Table:\n");
   for (i = 0; i < processes; i++)
{
      for (j = 0; j < resources; j++)
   {
            printf("\t%d", current[i][j]);
      }
   printf("\n");
   }

   printf("\nThe Maximum Claim Table:\n");
   for (i = 0; i < processes; i++)
{
      for (j = 0; j < resources; j++)
   {
         printf("\t%d", maximum_claim[i][j]);
      }
      printf("\n");
   }

   for (i = 0; i < processes; i++)
{
      for (j = 0; j < resources; j++)
   {
            allocation[j] += current[i][j];
      }
   }

   printf("\nAllocated resources:");
   for (i = 0; i < resources; i++)
{
      printf("\t%d", allocation[i]);
   }

   for (i = 0; i < resources; i++)
{
      available[i] = maxres[i] - allocation[i];
}

   printf("\nAvailable resources:");
   for (i = 0; i < resources; i++)
{
```

```
        printf("\t%d", available[i]);
    }
    printf("\n");

    while (counter != 0)
{
        safe = 0;
        for (i = 0; i < processes; i++)
    {
            if (running[i])
        {
                exec = 1;
                for (j = 0; j < resources; j++)
            {
                    if (maximum_claim[i][j] - current[i][j] > available[j])
            {
                    exec = 0;
                    break;
                    }
                }
                if (exec)
            {
                    printf("\nProcess%d is executing\n", i + 1);
                    running[i] = 0;
                    counter--;
                    safe = 1;

                    for (j = 0; j < resources; j++)
            {
                    available[j] += current[i][j];
                    }
                break;
                }
            }
        }
        if (!safe)
    {
            printf("\nThe processes are in unsafe state.\n");
            break;
        }
    else
    {
            printf("\nThe process is in safe state");
            printf("\nAvailable vector:");
```

```
            for (i = 0; i < resources; i++)
        {
                printf("\t%d", available[i]);
            }

          printf("\n");
        }
      }
system(“pause”);
}
```

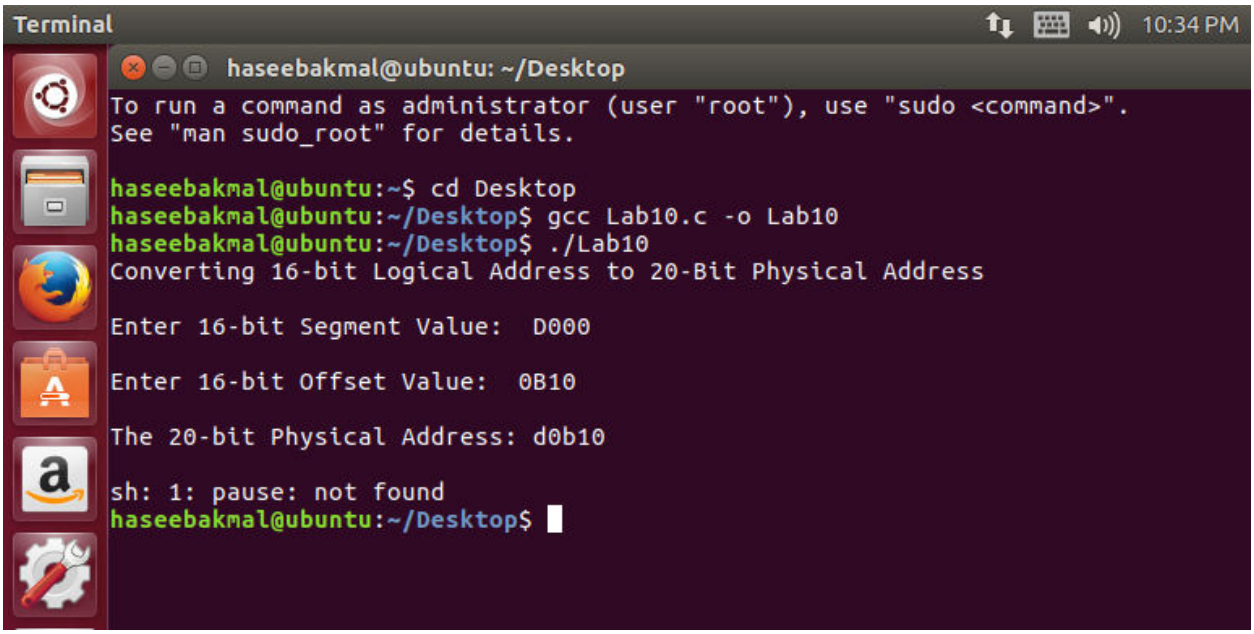
Mechanism to Conduct Lab:

Students and teacher will communicate through Skype/Adobe Connect. Student will edit, compile and run the C code for a given problem in Linux environment and share the screen with teacher.

Lab 10: Logical to Physical Address Translation in C

Write a c program to perform logical address to physical address translation. An address generated by the CPU is commonly referred to as a logical address, where as an address seen by the memory unit–that is, the one loaded into the memory-address register of the memory is commonly referred to as the physical address. In essence, logical data refers to an instruction or data in the process address space where as the physical address refers to a main memory location where instruction or data resides.

The output of program is as shown below:




```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int seg, off, logAdd, phyAdd;

        printf("Converting 16-bit Logical Address to 20-Bit Physical Address\n");
        printf("\nEnter 16-bit Segment Value:  ");
        scanf("%x",&seg);
        printf("\nEnter 16-bit Offset Value:  ");
        scanf("%x",&off);

        seg = seg*16;
        phyAdd = (seg + off);
```

```
        printf("\nThe 20-bit Physical Address: %x", phyAdd);

system("pause");
}
```

Mechanism to Conduct Lab:

Students and teacher will communicate through Google meet. Student will edit, compile and run the C code for a given problem in Linux environment and share the screen with teacher.

Lab 11: Calculating Paging Parameters and Paging Performance in C

11.1 Calculating Paging Parameters

Write a c program to calculate various paging parameters. Paging is a memory management scheme that permits the physical address space of a process to be noncontiguous. It avoids the considerable problem of fitting the various sized memory chunks onto the backing store, from which most of the previous memory-management schemes suffered.

The output of program is as shown below:

Enter Logical Address Space Information: 16
Enter number of pages: 16
Enter total memory: 2048
Enter frames: 32

No. of bits needed for p: 4
No. of bits needed for f: 5
No. of bits needed for d: 11
Logical address size: 15
Physical address size: 16

Solution:

```
#include<stdlib.h>
#include <stdio.h>
 #include <math.h>

int main()
{
double pages, mem, phyFrames, p, f , d, las, pas;
   printf("Enter Logical Address Space Information");
   printf("\nEnter number of pages: ");
      scanf("%lf", &pages);

   printf("\nEnter total memory: ");
      scanf("%lf", &mem);

   printf("\nEnter frames: ");
      scanf("%lf", &phyFrames);

     p = log2(pages);
    printf("\nNo. of bits needed for p: %lf", ceil(p));
```

```
    f = log2(phyFrames);
    printf("\nNo. of bits needed for f: %lf", ceil(f));

    d = log2(mem);
    printf("\nNo. of bits needed for d: %lf", ceil(d));

    las = p + d;
    printf("\nLogical address size: %lf", las);

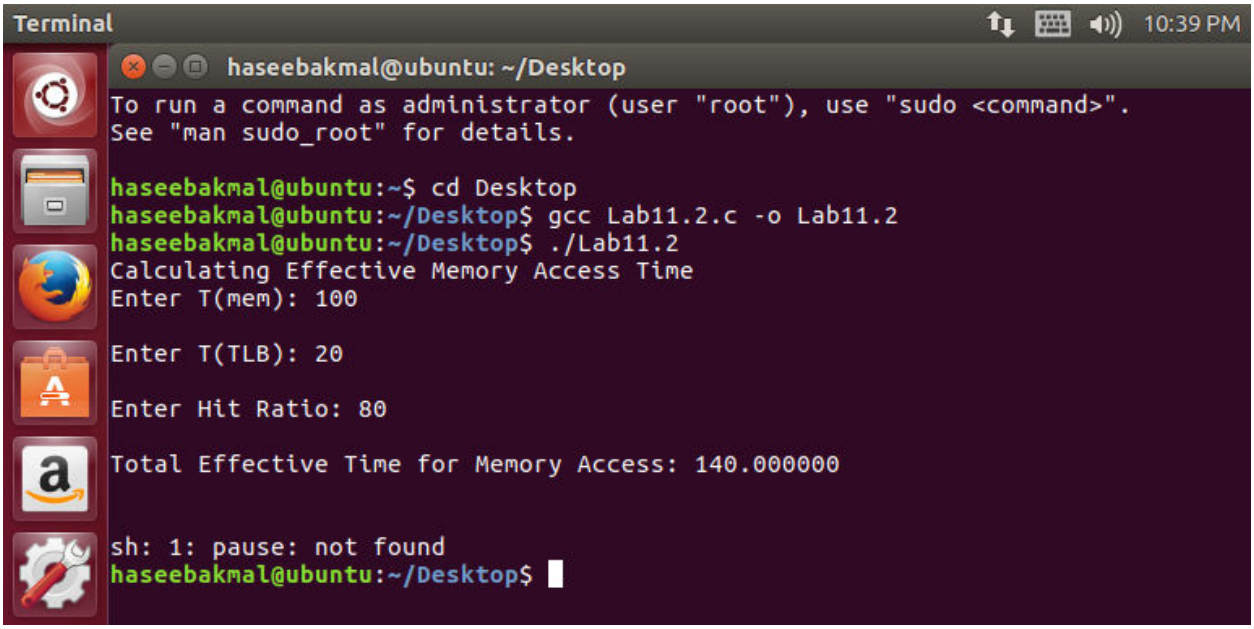
    pas = f + d;
    printf("\nPhysical address size: %lf", pas);

  printf("\n\n\n");
system("pause");
}
```

11.2 Calculating Paging Performance

Write a c program to calculate paging performance. The performance measure of paging is the effective memory access time based upon several parameters.

The output of program is as shown below:



Solution:

```
#include<stdlib.h>
#include <stdio.h>
 #include <math.h>
int main()
{
double Tmem, Ttlb, hRatio, tEffective, mRatio;
   printf("Calculating Effective Memory Access Time");

   printf("\nEnter T(mem): ");
      scanf("%lf", &Tmem);

   printf("\nEnter T(TLB): ");
      scanf("%lf", &Ttlb);

   printf("\nEnter Hit Ratio: ");
      scanf("%lf", &hRatio);

    mRatio = 100 - hRatio;

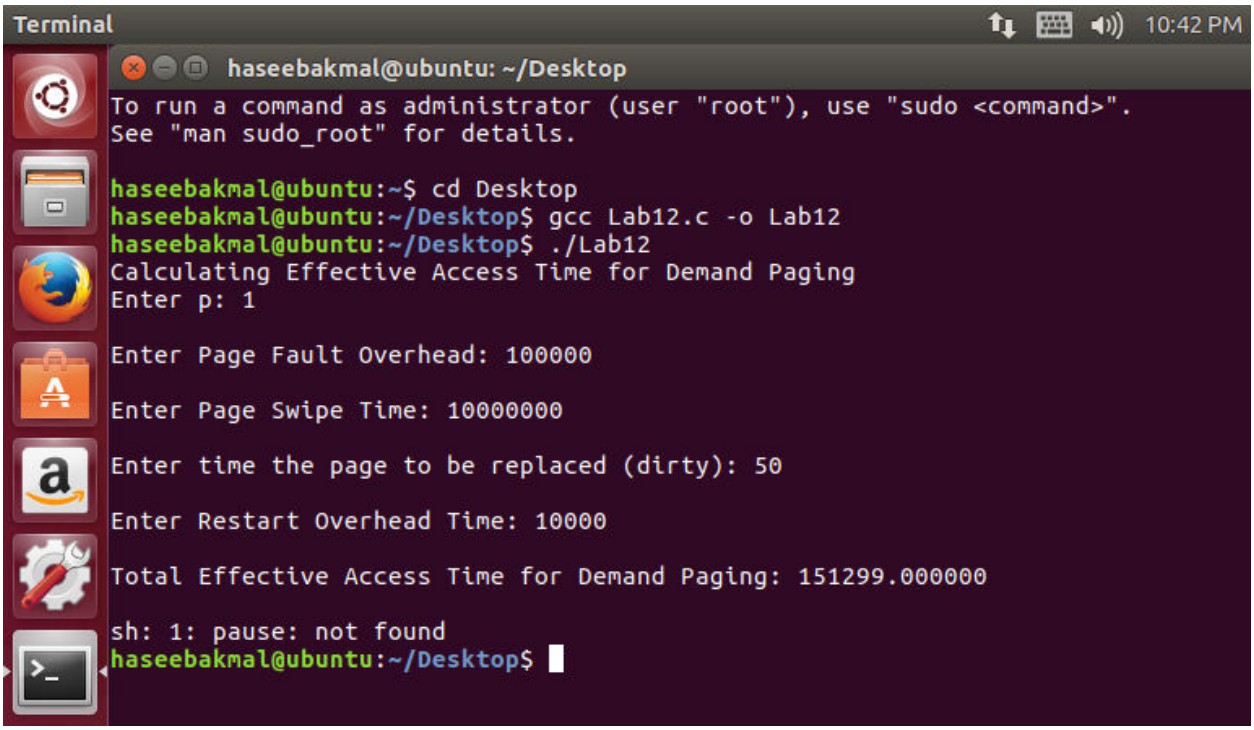
    tEffective =  (hRatio/100)*(Ttlb+Tmem)+ (mRatio/100) * (Ttlb + 2*Tmem);
        printf("\nTotal Effective Time for Memory Access: %lf", tEffective);

   printf("\n\n\n");
system("pause");
}
```

Lab 12: Demand Paging Performance Program in C

Write a c program to calculate demand paging performance. Demand paging can have a significant effect on the performance of a computer system. To see why, let us compute the effective access time for a demand paged memory. For most computer systems, the memory access time, denoted ma now ranges from 10 to 200 nanoseconds. As long as we have no page faults, the effective access time is equal to the memory access time. If, however a page fault occurs, we must first read the relevant page from disk, and then access the desired word.

The output of program is as shown below:




```
#include<stdlib.h>
#include <stdio.h>
int main()

{

double p, pfo, pst, dirty, ro, eat;

  printf("Calculating Effective Access Time for Demand Paging");

   printf("\nEnter p: ");

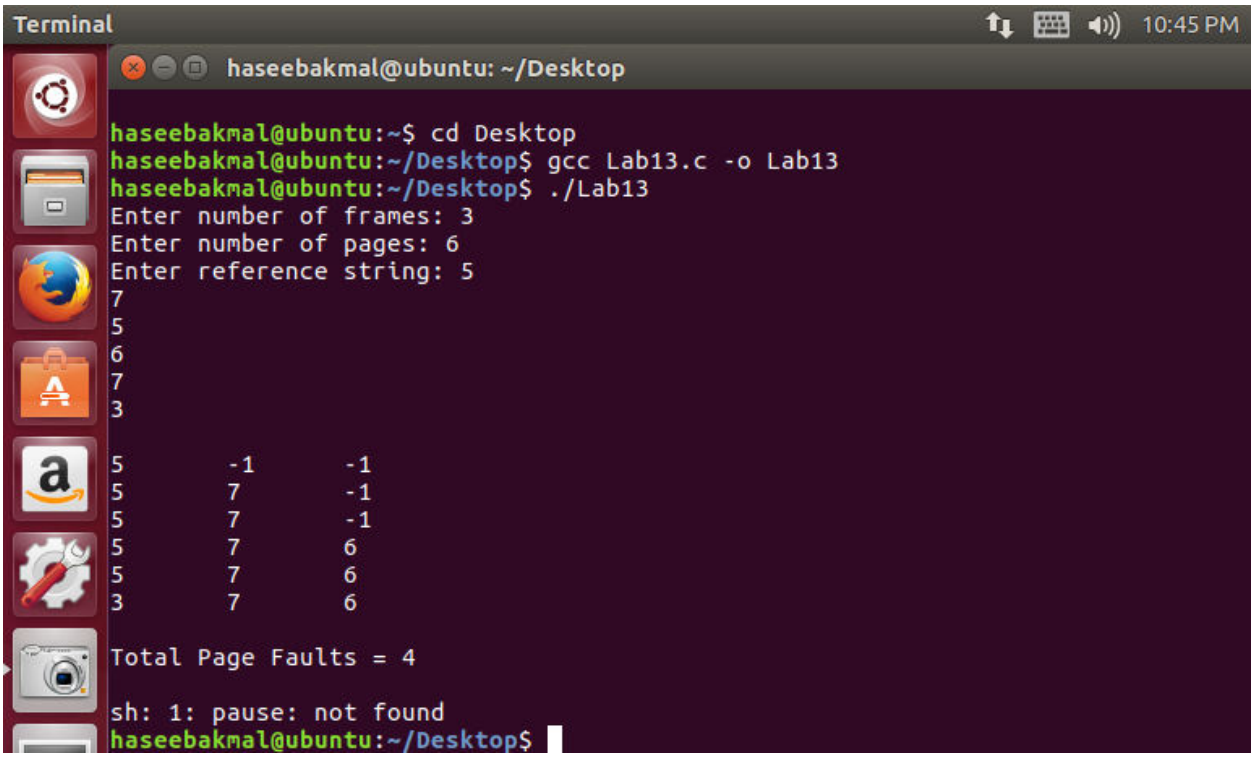
      scanf("%lf", &p);
```

```
        p = p / 100;
      printf("\nEnter Page Fault Overhead: ");
        scanf("%lf", &pfo);
    printf("\nEnter Page Swipe Time: ");
        scanf("%lf", &pst);
    printf("\nEnter time the page to be replaced (dirty): ");
        scanf("%lf", &dirty);
        dirty = dirty / 100;
    printf("\nEnter Restart Overhead Time: ");
        scanf("%lf", &ro);
          eat =  100*(1-p) +   ( pfo + 2*ro + (dirty*pst) + (dirty*2*pst) ) * p ;
              printf("\nTotal Effective Access Time for Demand Paging: %lf", eat);
system("pause");
}
```

Lab 13: Implementing Least Recently Used (LRU) Page Replacement Algorithm in C

Write a c program to implement LRU Page Replacement algorithm. If we use the recent past as an approximation of the near future, then we will replace the page that has not been used for the longest period of time. Least Recently Used (LRU) page replacement algorithm basically works on the concept that the pages that are heavily used in previous instructions are likely to be used heavily in next instructions. The page that are used very less are likely to be used less in future.

The output of program is as shown below:




```
#include<stdio.h>
#include<stdlib.h>
 int findLRU(int time[], int n){
   int i, minimum = time[0], pos = 0;
    for(i = 1; i < n; ++i){
      if(time[i] < minimum){
         minimum = time[i];
         pos = i;
      }
   }
    return pos;
```

```
}
int main() {
  int no_of_frames, no_of_pages, frames[10], pages[30], counter = 0, time[10], flag1, flag2, i, j,
pos, faults = 0;
   printf("Enter number of frames: ");
   scanf("%d", &no_of_frames);

   printf("Enter number of pages: ");
   scanf("%d", &no_of_pages);

   printf("Enter reference string: ");
   for(i = 0; i < no_of_pages; ++i){
      scanf("%d", &pages[i]);
   }

   for(i = 0; i < no_of_frames; ++i){
      frames[i] = -1;
   }

   for(i = 0; i < no_of_pages; ++i){
      flag1 = flag2 = 0;

      for(j = 0; j < no_of_frames; ++j){
         if(frames[j] == pages[i]){
            counter++;
            time[j] = counter;
               flag1 = flag2 = 1;
               break;
            }
      }

      if(flag1 == 0){
         for(j = 0; j < no_of_frames; ++j){
            if(frames[j] == -1){
               counter++;
               faults++;
               frames[j] = pages[i];
               time[j] = counter;
```

```
                flag2 = 1;
                break;
            }
        }
    }

    if(flag2 == 0){
        pos = findLRU(time, no_of_frames);
        counter++;
        faults++;
        frames[pos] = pages[i];
        time[pos] = counter;
    }
    printf("\n");

    for(j = 0; j < no_of_frames; ++j){
        printf("%d\t", frames[j]);
    }
  }

  printf("\n\nTotal Page Faults = %d", faults);

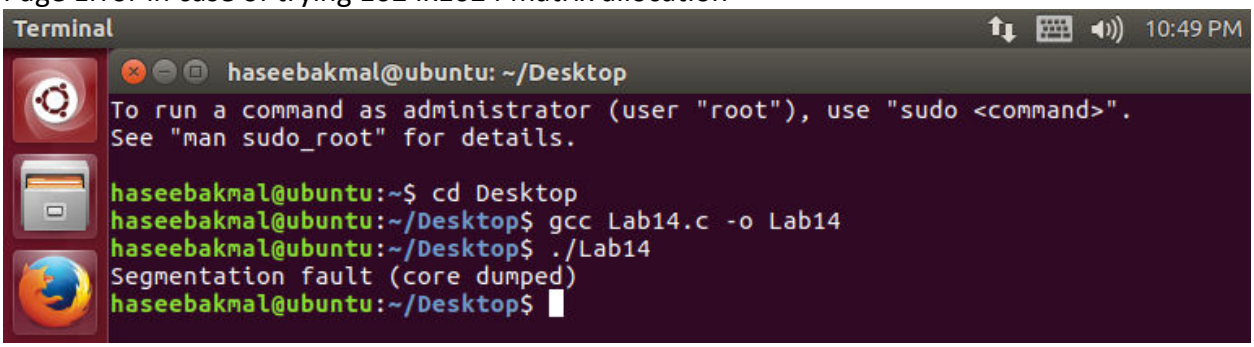
system("pause");
}
```

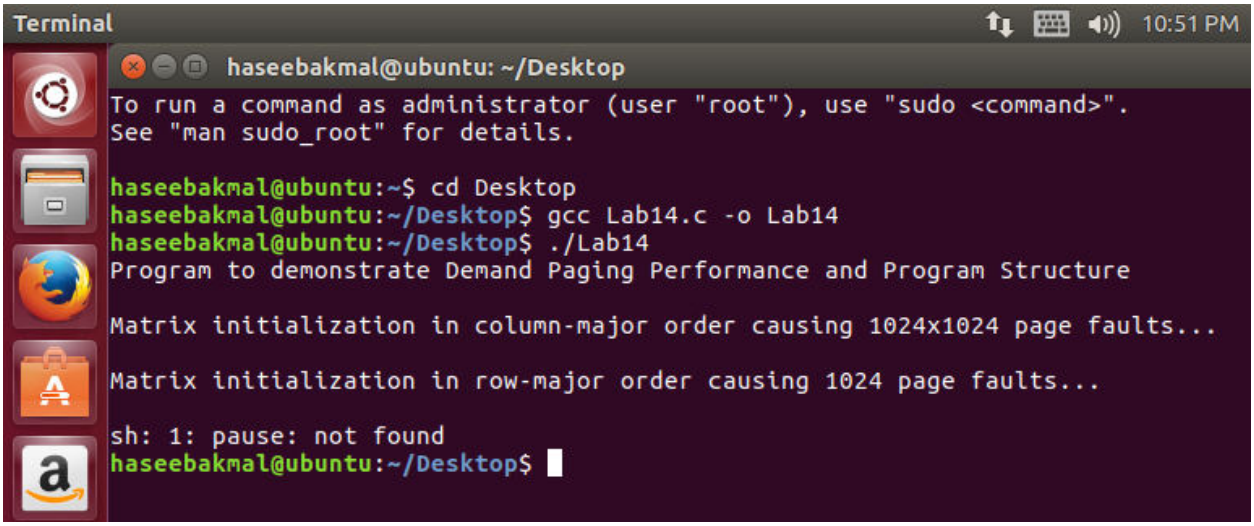
Lab 14: Demand Paging and Program Structure in C

Write a c program to declare and initialize two 2-Dimensional arrays (or matrices); one by column-major order and other by row-major order. Demand paging is designed to be transparent to the user program. However, in some cases system performance can be improved if the programmer has an awareness of the underlying demand paging and execution environment of the language used in the program.

The output of program is as shown below:

Page Error in case of trying 1024x1024 matrix allocation






```
#include<stdio.h>
#include<stdlib.h>
int main()
{
printf("Program to demonstrate Demand Paging Performance and Program Structure");
// Code 1: Column-major order causing 1024x1024 page faults
int A[1024][1024];
```

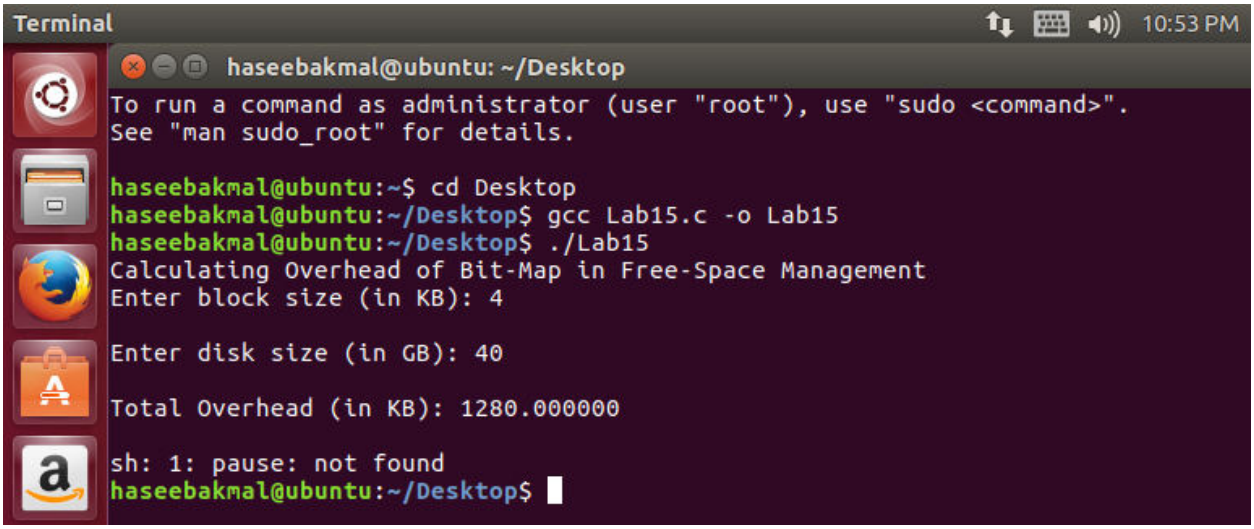
```
   for (int j=0; j<1024; j++){
      for (int i=0; i<1024; i++){
         A[i][j] = 0;
         }
       }
printf("\n\nMatrix initialization in column-major order causing 1024x1024 page faults...");

// Code 2: Row-major order causing 1024 page faults only
int B[1024][1024];
   for (int i=0; i<1024; i++){
      for (int j=0; j<1024; j++){
            B[i][j] = 0;
            }
      }
printf("\n\nMatrix initialization in row-major order causing 1024 page faults...");
printf("\n\n");
system("pause");
}
```

Lab 15: Calculating Bit Map Overhead in Disk Free-Space Management in C

Write a c program to calculate the Overhead of Bit Map (or Bit Vector) in Free-Space Management. Frequently, the free space list is implemented as a bit map or bit vector. Each block is represented by 1 bit. If the block is free, the bit is 1;if it is allocated, the bit is 0. This approach is relatively simple and efficient in finding the first free block or n consecutive free blocks on the disk.

The output of program is as shown below:




```
#include<stdlib.h>
#include <stdio.h>
int main()
{
double blockSize, diskSize, overhead;
printf("Calculating Overhead of Bit-Map in Free-Space Management");
        printf("\nEnter block size (in KB): ");
        scanf("%lf", &blockSize);

        printf("\nEnter disk size (in GB): ");
        scanf("%lf", &diskSize);

        diskSize = diskSize*1024*1024;
        overhead = (diskSize / blockSize)/(8*1024);

        printf("\nTotal Overhead (in KB): %lf", overhead);
```

```
printf("\n\n");
system("pause");
}
```

Problem Statement Lab 16

Write the C program to create a basic simulation of a disk space management system. The program should manage disk space allocation and de-allocation for files stored on a simulated disk represented by an array. The system needs to allocate contiguous blocks of free space for new files and mark them as allocated. Additionally, it should be able to de-allocate space when files are deleted, marking the corresponding blocks as free for reuse. The program should simulate file creation and deletion by allocating and deallocating space accordingly, with print statements indicating the allocation and de-allocation of disk blocks for each file operation.

Solution:

```
#include <stdio.h>

#define TOTAL_BLOCKS 100 // Total number of disk blocks

int disk[TOTAL_BLOCKS] = {0}; // Array to represent disk blocks (0: free, 1: allocated)

// Function to allocate space for a new file
void allocateSpaceForFile(int fileSize) {
    int startBlock = -1;
    int consecutiveBlocks = 0;

    for (int i = 0; i < TOTAL_BLOCKS; i++) {
        if (disk[i] == 0) {
            if (startBlock == -1) {
                startBlock = i;
            }
            consecutiveBlocks++;
            if (consecutiveBlocks == fileSize) {
                for (int j = startBlock; j < startBlock + fileSize; j++) {
                    disk[j] = 1;
                }
                printf("Allocated space for file of size %d starting at block %d\n", fileSize, startBlock);
                return;
            }
        } else {
            startBlock = -1;
            consecutiveBlocks = 0;
        }
    }

    printf("Error: Not enough free space to allocate for file of size %d\n", fileSize);
}

// Function to deallocate space when a file is deleted
void deallocateSpaceForFile(int startBlock, int fileSize) {
```

```
    for (int i = startBlock; i < startBlock + fileSize; i++) {
        disk[i] = 0;
    }
    printf("Deallocated space for file starting at block %d\n", startBlock);
}

int main() {
    // Simulate file creation and deletion
    allocateSpaceForFile(20);
    allocateSpaceForFile(30);
    deallocateSpaceForFile(0, 20);
    allocateSpaceForFile(10);

    return 0;
}
```

Program compiled through online C compiler tool.

```
/tmp/KC6KBYKQKH.o
Allocated space for file of size 20 starting at block 0
Allocated space for file of size 30 starting at block 20
Deallocated space for file starting at block 0
Allocated space for file of size 10 starting at block 0
```